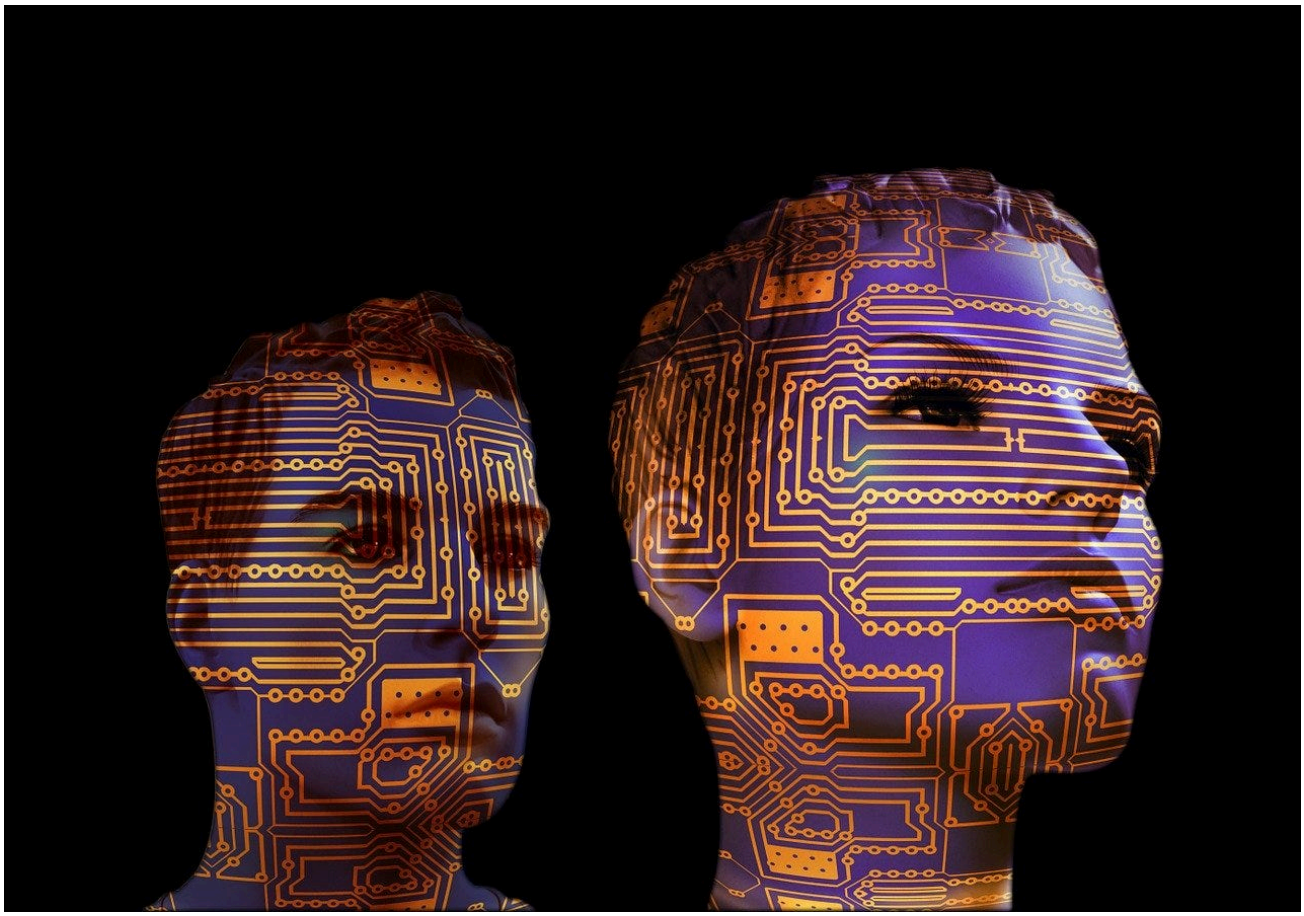
ANALYTICS

Reaching for the gut of Machine Learning: A brief intro to CLT

Knowing the fundamentals of the computational learning theory can empower you immensely as a practitioner of machine learning.

Tirthajyoti Sarkar
Oct 26, 2018 12 min read



Introduction

Suppose, it is a sunny day, you have friends visiting and your favorite restaurant opened a branch – 12 miles away. Generally,

you avoid long drives, but would to go out for lunch today? Do you have past examples of this kind of situation (some factors are positive and some are negative) from which you have formulated a rule?

This is how we learn from past experiences and actions, form rules, and apply them to present situations. Machines are no different either. But there is also a theory behind this kind of machine learning.

Computational **L**earning **T**heory (**CLT**) is a branch of statistics/machine learning/artificial intelligence (in general) which deals with fundamental bounds and theorems about analyzing our (man and machine) ability to learn rules and patterns from data.

It **goes beyond the realm of specific algorithms** that we regularly hear about – *regression, decision trees, support vector machines* or *deep neural network* – and tries to answer **fundamental questions about the limits and possibilities of the whole enterprise of machine learning**.

> This is how we learn from past experiences and actions, form rules, and apply them to present situations. Machines are no different either

Sounds exciting? Please read on for a quick tour of this realm...

What are some of the fundamental questions?

When studying machine learning it is natural to wonder what general laws may govern machine (and non-machine) learners. For example,

- Is it possible to identify classes of learning problems that are inherently difficult or easy, independent of the learning algorithm?
- Can one characterize the **number of training examples necessary** or sufficient to assure successful learning?
- How is this number affected if the learner is allowed to pose queries to the trainer, versus observing a random sample of training examples?
- Can one characterize the **number of mistake**s that a learner will make before learning the target function?
- Can one characterize the inherent **computational complexity** of classes of learning problems?

> Here is the disappointing answer: General answers to all these questions are not yet known.

But we can focus on a particular setting which comes up most often in any practical machine learning task, **** that o**f – inductively learning an unknown target function, given only training examples of this target function and a set of candidate hypothese**s.

Within this setting, we are chiefly concerned with questions such as,

- *how many training examples are sufficient to successfully learn the target function*, and
- *how many mistakes will the learner make before succeeding*?

As we shall see, it is possible to set quantitative bounds on these measures, depending on attributes of the learning problem such as,

- the size or complexity of the hypothesis space considered by the learner
- the accuracy to which the target concept must be approximated
- the probability that the learner will output a successful hypothesis
- the manner in which training examples are presented to the learner

> CLT tries to answer **fundamental questions about the limits and possibilities of the whole enterprise of machine learning**.

Illustrations of the Keywords

For the sake of a comprehensive discussion, here is my attempt to define the basic keywords generally used in CLT.

Data: A given set of examples from which we are trying to learn a target concept.

Sunny?	Favorite cuisine restaurant distance?	Got friends?	Go out for lunch?
Yes	10 miles	Yes	Yes
No	2 miles	No	No
Yes	1 mile	Yes	Yes
Yes	8 miles	No	No
No	12 miles	Yes	No
No	3 miles	Yes	Yes

Target concept (aka rule): This is the hidden pattern we are trying to learn from the data for example – "We go out for lunch ***IF*** it is sunny ***AND*** we got friends ***OR IF*** favorite cuisine restaurant is nearby ***AND*** we got friends ***AND*** even IF it is ***NOT*** sunny"...

This kind of rule is written in terms of statements belonging to propositional logic i.e. Truth values expressed with AND, OR, NOT.

Hypothesis space: This is a set of hypotheses from which we hope to discover the target concept. In this example, the 'intelligent' choice of hypotheses should look like the logically connected phrase we wrote above as the target concept. But they could be simpler or different looking such as,

- IF it is sunny
- IF we got friends OR it is sunny

Note the subtle difference between the two hypotheses above and the target concept. **These two hypotheses are not rich enough to capture the true target concept** and they will have errors if applied to the given data.

Why?

Because they are not of the form "conjunction of disjunctive statements" i.e. (X1 & X2) | (X3 & X4) | (X5 & !X6). They are too simplistic and general and unable to capture all the nuance of the data presented.

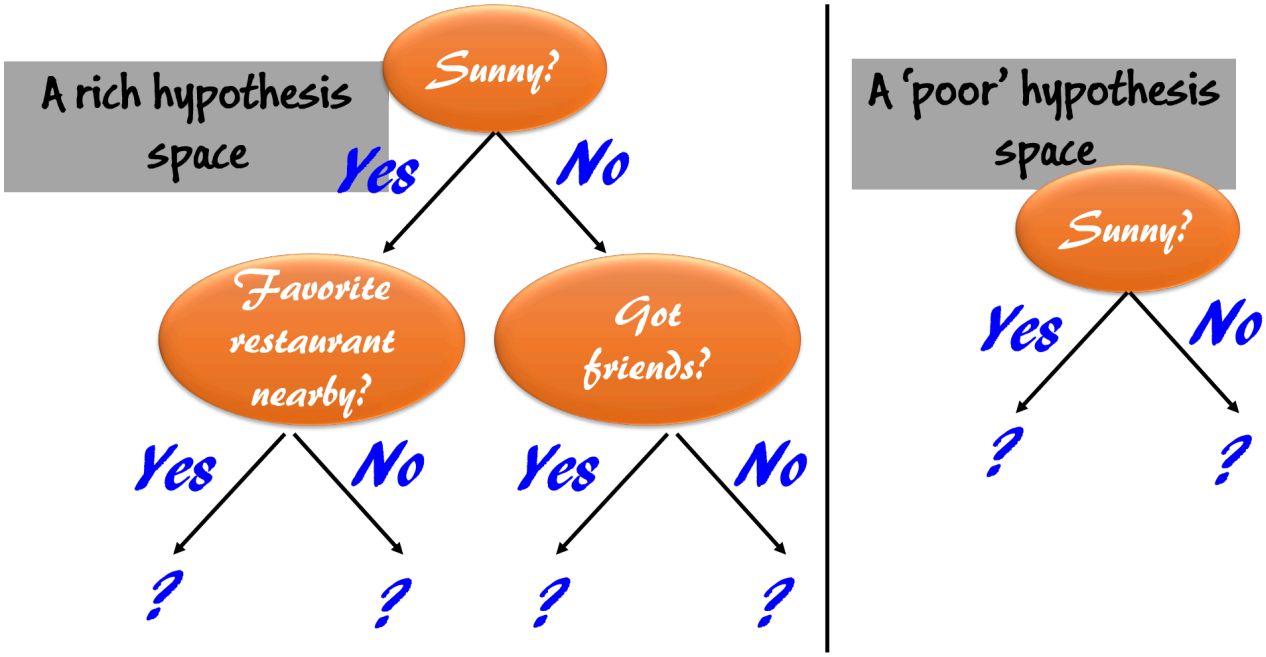
So, the moral of the story is that whether you will be successful in your search for target concept in a machine learning (here a classification) task, depends largely on the richness and complexity of the hypothesis space you choose to work with.

Predictor and response variables: These are self-explanatory. The '*Sunny*?', '*Favorite restaurant distance*' and '*Got friends?*' are predictor variables and 'Go out for lunch' is the response variable.

Feeling abstract? Need a concrete example?

Remember the decision trees for classification? Here are two such trees and their richness in terms of hypothesis space with regard to the learning problem described above.

If you read the conditions satisfied by the edges of the tree starting from the root and down to a leaf, you will automatically get propositional logic statements just like above. Try it.



Rich and 'poor' hypothesis space illustrations

> So, the moral of the story is that whether you will be successful in your search for target concept in a machine learning (here a classification) task, depends largely on the richness and complexity of the hypothesis space you choose to work with.

Size of the hypothesis space? There is a trap there!

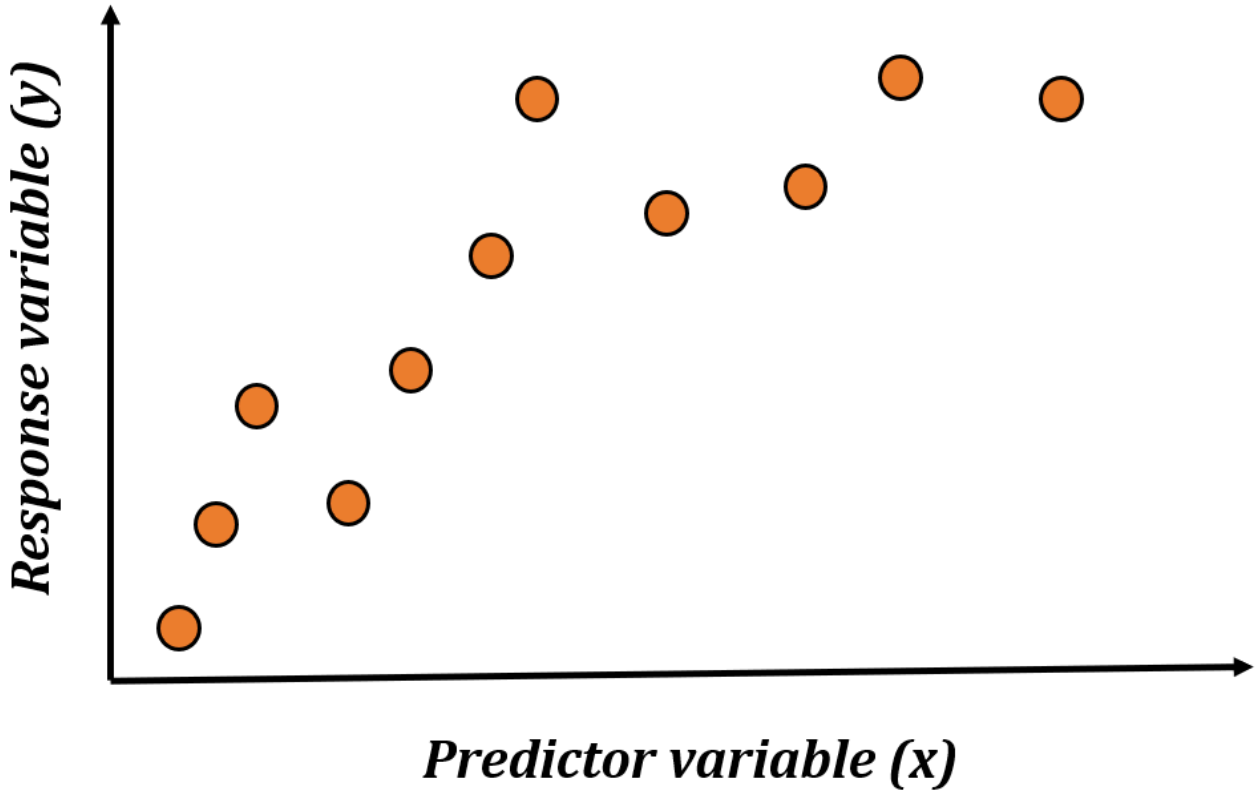
So, what is the size of the hypothesis space here?

If you are building trees with all three predictor variables you can start with any one of those, then you can have **O(2^(n-1))** leaves at the depth *n* if you are using all the predictors. But there is a choice of putting 'Yes' and 'No' on all those leaves. So, the size can be as large as O(n2^(2^n)). *For this example, with 3 variables, we can have 32^(2^(3–1)) = 48 trees!* These 48 trees each represent a different hypothesis. Many of them are superfluous and can be eliminated with simple thinking, but that is beyond the point.

So, if we consider a rich enough and big enough hypothesis space, are we guaranteed to find the target concept (given enough data)?

NO.

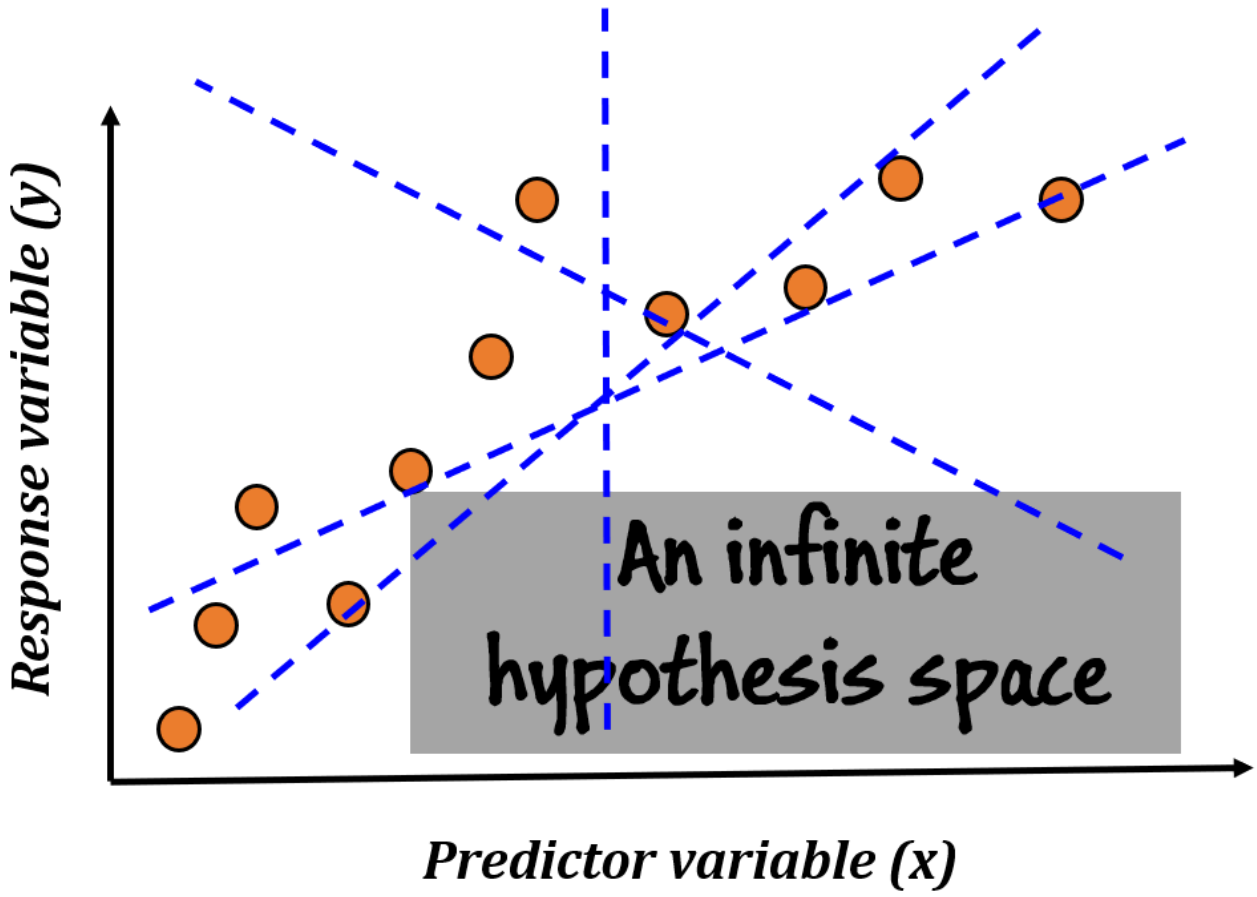
Consider a simple linear regression problem.



How many predictors are there? **1**. So, what is the hypothesis space size? 2? 4?

The answer is – **INFINITE**.

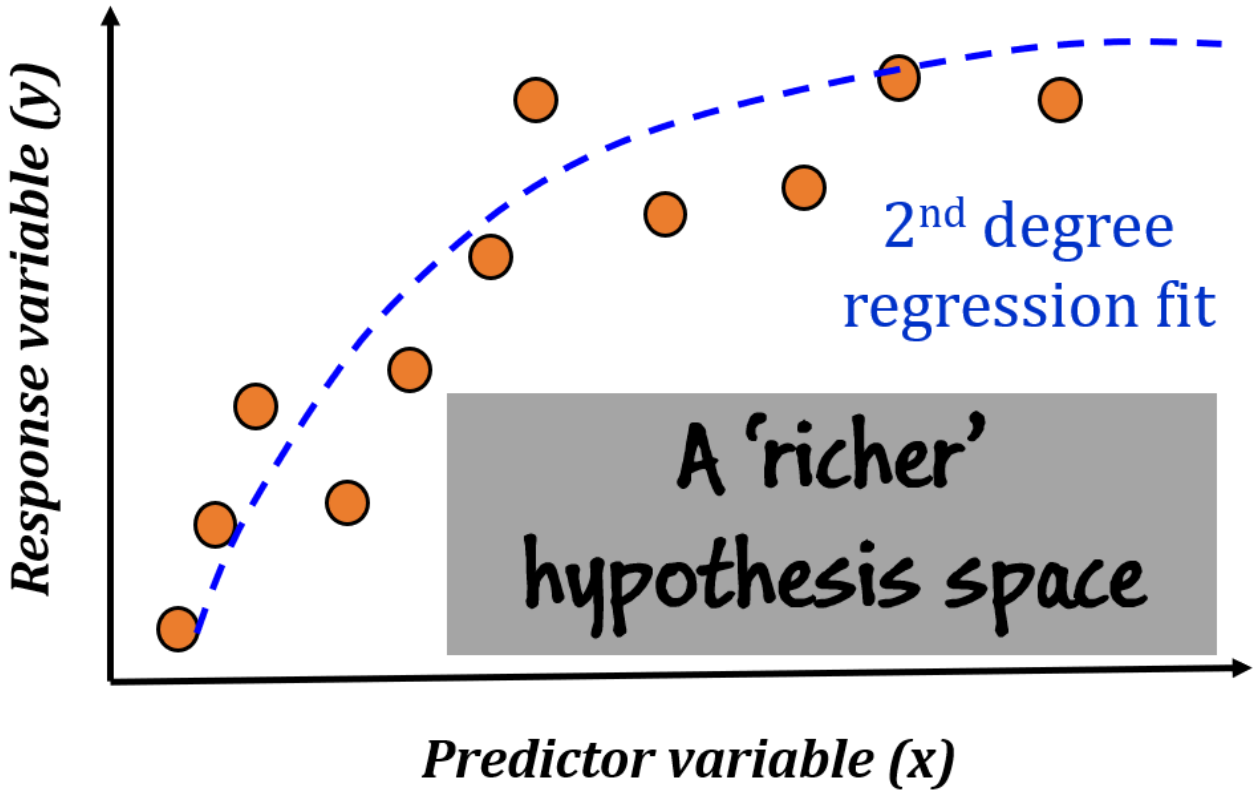
How many straight lines you can draw in a plane?



Did your respect for the innocuous **regression algorithm with least-square error minimization** just grow immensely? After all, it is able to find the optimal line from those infinite possibilities 🙂

> But here is the problem – even with an infinity-sized hypothesis space, we cannot hope to hit the true target concept in this regression task.

This is due to the fact that the **true concept lies in another space whose size is a 'bigger infinity'**. In this example, the true functional relationship between ***y*** and ***x*** may be a quadratic one. That implies infinite possibilities for the 2nd-degree terms + infinite possibilities for the linear terms + infinite possibilities of the intercept.



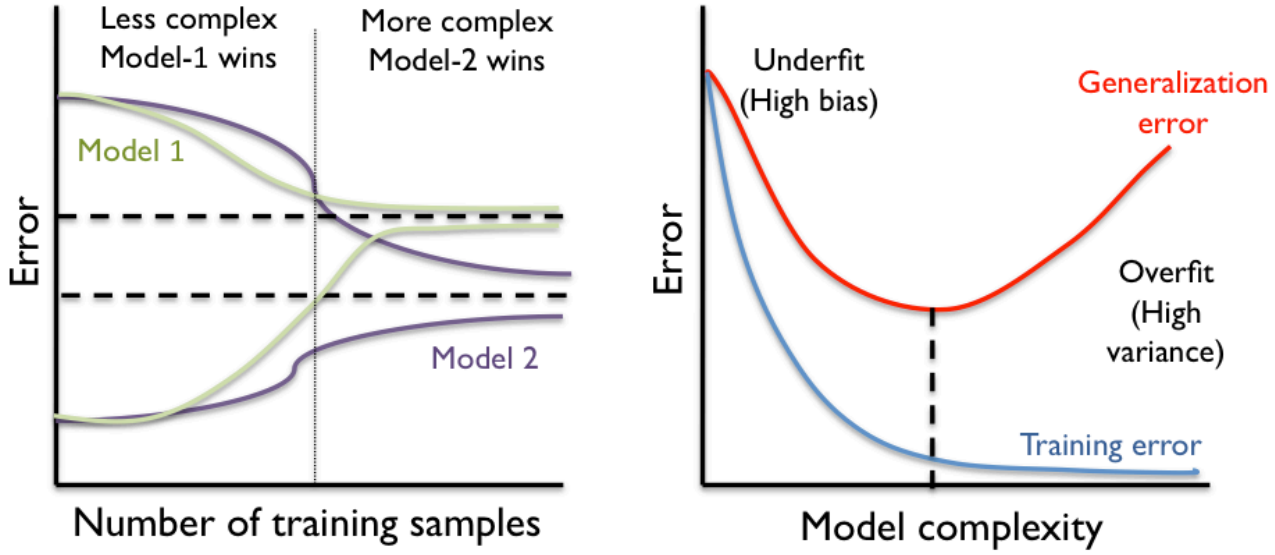
Unless we decide to include a 2nd degree term in our hypothesis space, we will never come close to finding the true function even with our infinity-sized space with only the linear term (and intercept).

> **So, in many cases, we need a richer space, not necessarily bigger.** And, like Thanos, we need to constantly worry and search which infinity-sized space could be the best for the particular dataset we have to learn from!

So, what are the key quantities CLT tries to estimate?

For the most part, CLT focuses not on individual learning algorithms, but rather on broad classes of learning algorithms characterized by the hypothesis spaces they consider, the presentation of training examples, etc. The key quantities it tries to reason about are,

- ***Sample complexity***: How many training examples are needed for a learner to converge (with high probability) to a successful hypothesis?
- ***Computational complexity***: How much computational effort is needed for a learner to converge (with high probability) to a successful hypothesis?
- ***Mistake bound***: How many training examples will the learner misclassify before converging to a successful hypothesis?



Model complexity and learning curve show how much data and what class of hypothesis space we need

There are various ways to specify what it means for the learner to be "successful." We might specify that to succeed, the learner must output a hypothesis identical to the target concept. Alternatively, we might simply require that it output a hypothesis that agrees with the target concept most of the time, or that it usually output such a hypothesis.

Similarly, we must specify how training examples are to be obtained by the learner. It is a possibility that training examples are presented by a helpful teacher, or obtained by the learner performing carefully planned experiments (**think A/B testing or scientific experiments**), or simply generated at random according to some natural process outside the learner's control (**think random click streams, cancer cell's interaction with drug**). As we might expect, the answers to the above questions depend on the particular setting, or learning model, we have in mind.

The Amazing World of "Probably Approximately Correct" (PAC)

In this theoretical setting, we assume that all observed data was generated from a unknown but fixed (not time varying)

stribution process whose parameters do not get affected by

the process of our drawing sample from it. Also, without loss of generality, we can assume noise-free measurement process.

Let ***H*** be the hypothesis space, ***D*** be the unknown true distribution, and ***c*** be the target concept. Let us say that after observing some data ***d***, the learning algorithm ***L*** outputs a hypothesis ***h*** which it thinks is the best approximation of ***c***.

What is the measure of error/success here? Error is where the prediction of ***c*** and ***h*** disagrees.

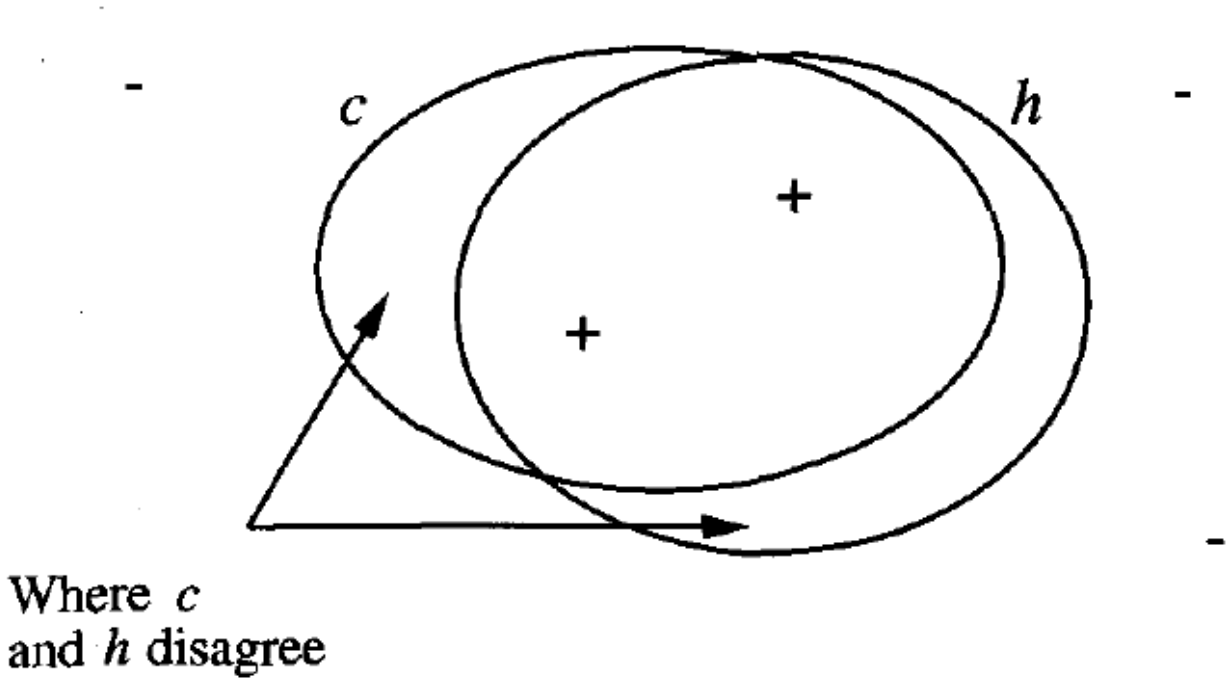


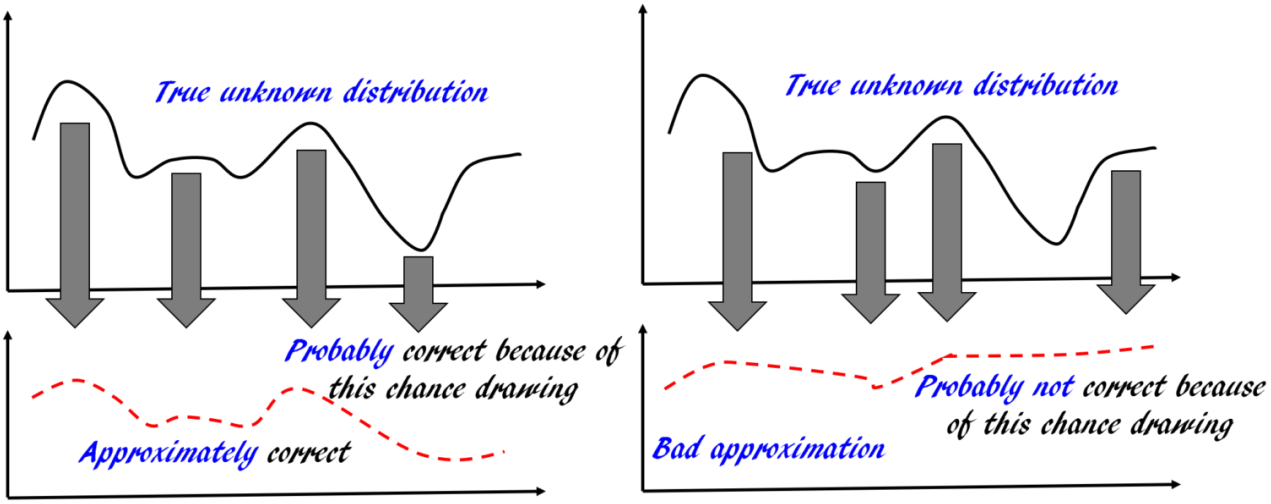
Image credit: "Machine Learning", by Tom Mitchell

> Our aim is to characterize classes of target concepts that can be **reliably learned** from a **reasonable number** of randomly drawn training examples and a **reasonable amount of computation**.

We don’t have infinite time, compute power, storage, or sensor bandwidth. Therefore, we don’t try to make the error zero. For that, we need to see every instance possible as generated by ***D***.

For the same reason, we restrict ourselves to examining limited training samples and using a polynomial-time algorithm.

Moreover, given that the training samples are drawn randomly, there will always be some nonzero probability that the training examples encountered by the learner will be misleading.



Therefore, we will require only that its **error be bounded** by some constant, **that can be made arbitrarily small**. Moreover, we will not require that the learner succeed for every sequence of randomly drawn training examples, instead, our demand will only be that its **probability of failure be bounded by some constant, that can be made arbitrarily small**.

> In short, we require only that the learner ***probably learn*** a hypothesis that is ***approximately correct***.

With these settings, we say that our concept class ***C***, to which the target concept ***c*** belong, is **PAC-learnable** if the learning algorithm can output the hypothesis ***h*** after being trained with data ***d*** with a certain high ***probability*** and with a certain low ***error*** with a computational effort which is a polynomial function of 1/***error***, 1/(1-***probability***), size of ***C*** and length of the data ***d***.

ample complexity bound

We will not go through the formal proof here, but following the concepts, discussed in the previous section, an amazingly simple and elegant mathematical bound can be derived to answer the following question,

> "What is the minimum number of training sample needed to learn an approximately correct hypothesis with certain probability when the hypothesis space has a certain size?"

The bound is given by,

$$m \geq \frac{1}{2\epsilon^2}(\ln|H| + \ln(1/\delta))$$

m = number of training examples

ϵ = error

δ = probability of mistake

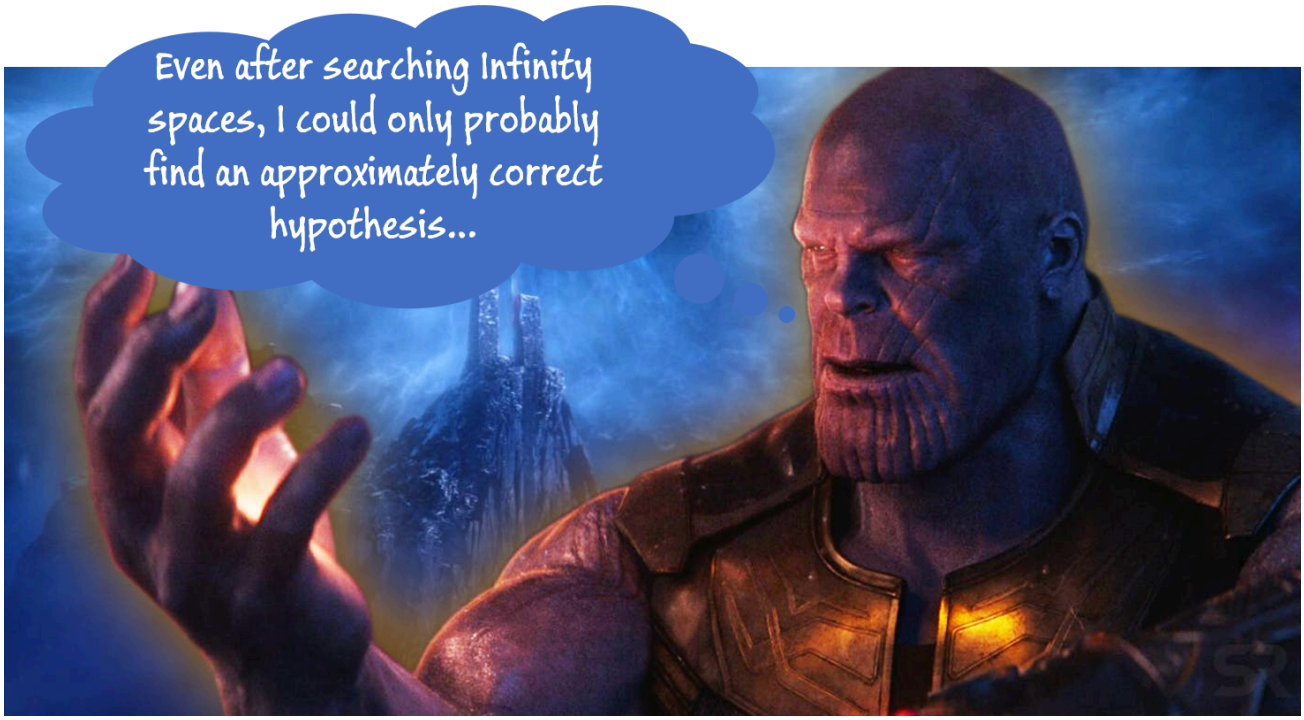
$|H|$ = size of the hypothesis space

This is called **Haussler bound**. This inequality readily demonstrates the following facts which we experience in practice for any machine learning task,

- **we need higher number of training data to reduce test/generalization error**
- **the algorithm needs to 'see' a higher number of training examples to increase its confidence in learning i.e. reduce**

the probability of mistake

- **a richer and bigger hypothesis space means a larger dimension to search through in the quest of the correct hypothesis, and that needs higher number of training examples**



Our short tour comes to an end here but much more 'learning' lie ahead

There is a general criticism of this type of complexity bounds that they are too pessimistic and do not represent the practical learning problems. There is much debate on that topic and you can read some of it from the references I have put at the end of this article.

A natural question arises about infinite hypothesis spaces. Does the sample bound go to infinity? It turns out that there is a concept called 'VC-dimension' (V here stands for Vladimir Vapnik, the inventor of support vector machine), which helps to keep the sample complexity finite for many practical learning problems even with infinite hypothesis spaces like a linear regression or a

kernel-machine. But we could probably talk about such advanced concepts in another article.

I hope this article could initiate some interesting concepts about fundamental machine learning theory and helped to wheat your appetite to learn more on these topics.

For an excellent tutorial on these topics, watch this video,

Ali Ghodsi, Lec 19: PAC Learning

Data Science Courses

Watch on

References to consult

1. "Machine Learning", Tom Mitchell (1997).
2. PAC learning tutorial from CMU
3. "PAC learning, VC-dimensions, and Margin based bounds"
4. "PAC learning is undecidable"
5. "Efficiency and Computational Limitations of Learning Algorithms"